Lab 3 - EF, Routing

Predaja: 8.5.

Zadaci i bodovanje

Kriterij	Bodovi
Prilagodba modela za EF (anotacije, veze), konfiguracija EF	1
Razumijevanje EF principa (usmeno ispitivanje)	1
Razumijevanje routing principa (usmeno ispitivanje)	1
Izrada semantičkog i routing modela (md file) pomoću AI	1
Izrada i korištenje skill-ova (EF skill, Routing skill, UX skill)	1

Nužni uvjeti za predaju vježbe

- [ ] Konfigurirati EF u projektu
  - [ ] Dodati ispravne anotacije na model kako bi bio EF-ready
  - [ ] Podesiti virtual i ICollection<> svojstva kako bi veze među klasama/tablicama bile ispravne
  - [ ] Instalirati bazu podataka (MSSQL ili neku drugu), ili ju pokrenuti u Docker-u, podesiti connection string
  - [ ] Podesiti DbContext i osigurati potrebne DI
- [ ] Prebaciti app s mock repository na EF repository
  - [ ] Generirati inicijalnu migracijsku skriptu
- [ ] Podesiti i proučiti prilagodbu usmjeravanja na barem 4 akcije controller-a po želji (custom routing, default se ne broji)
- [ ] Izrada semantičkog DB modela: semantic-model.md
  - [ ] Sadržaj: sažeti popis modela/klasa/tablica, popis glavnih svojstava i veze među tablicama/klasama
- [ ] Izrada semantičkog modela usmjeravanja: sitemap.md
  - [ ] Za svaki dostupni URL u aplikaciji, treba pisati koji je controller, koja akcija i koji view-ovi se koriste

☐ Konfiguracija skill-ova: https://code.visualstudio.com/docs/copilot/customization/agent-skills (napraviti barem jedan skill od dolje navedenih je dovoljno; ili neki drugi koji bolje odgovara projektu)

- Konfiguracija SKILL.md datoteke vezano za entity-framework skill - koristi se kad treba dodati izmjenu u neku EF klasu, generirati migraciju, itd.
- Konfiguracija SKILL.md datoteke vezano za izradu "list" stranice - koristi se kad trebamo napraviti "list" stranicu. Napraviti novu list stranicu koristeci skill za primjer (proširiti app)
- Konfiguracija SKILL.md datoteke vezano za izradu "edit form" stranice - koristi se kad trebamo napraviti edit/create stranicu. Napraviti novu edit formu koristeći skill za primjer (proširiti app)

URL usmjeravanja (routing)

Dosad se koristio samo osnovni oblik routinga - /Controller/Action/{id – opcionalno}. U nastavku će se razmotriti kompleksniji i prilagođeniji scenariji za rukovanje URL usmjeravanjima kako bi aplikacija bolje odgovarala korisnicima.

Program.cs

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Pogledajmo gornju jedinu definiranu rutu – osnovnu rutu (otisnuto masnim slovima). Slijede objašnjenja pojedinih parametara:

- **name** – jedinstveni naziv rute, koristi se kako bi svaka definicija rute imala jedinstveno ime
- **pattern** – url shema preko koje se aktivira ruta. Primjerice, gornja ruta definira da se URL shema može sastojati od najviše 3 dijela (recimo HOST/Xxx/Yyy/123), gdje se iz prvog parametra (Xxx) iščitava naziv odgovarajućeg Controller-a – u ovom slučaju to bi bila klasa XxxController; iz drugog parametra se iščitava naziv odgovarajuće akcije u Controlleru – to bi bila akcija (ActionResult Yyy(...) { }); a iz trećeg dijela se izvlači parametar koji se prosljeđuje akciji (ActionResult Yyy(int id), gdje parametar

„id" poprima vrijednost 123). Id nije nužno cijeli broj – može biti i string, iako u većini aplikacija se iza imena id očekuje cijeli broj.

- **Inicijalne vrijednosti** – ako URL putanja nije u potpunosti definirana, tada se koriste inicijalne (ili fallback) vrijednosti. Primjerice, za controller, fallback vrijednost je Home. Za parametar action, fallback vrijednost je Index.
  - **URL**: `HOST/Xxx/Yyy/123` – inicijalne vrijednosti nemaju nikakvog efekta jer su svi parametri zadani. Poziva se controller = XxxController, akcija = Yyy, paramter id = 123.
  - **URL**: `HOST/Xxx/Yyy` – treći parametar je opcionalan, stoga se Id jednostavno nigdje ne pridjeljuje
  - **URL**: `HOST/Xxx` – u ovom slučaju definiran je XxxController, a zaključuje se da ako drugi parametar nije specificiran, podrazumjeva se akcija Index.
  - **URL**: `HOST/` - u ovom slučaju niti jedan parametar nije definiran, te se podrazumjeva akcija Index i controller Home.
- **defaults** – dodatni parametar funkcije MapControllerRoute je defaults. Ukoliko se iz URL-a ne može odrediti koji controller i koja akcija obrađuje web zahtjev, tada se iz defaults parametra preuzimaju tražene vrijednosti.

Važna napomena: nomenklatura u ASP.NET MVC je veoma bitna, i treba poštivati određena pravila. Ukoliko je potrebno, moguće je ta pravila zaobići i prilagoditi potrebama, no to je preporučljivo samo u iznimnim situacijama. Evo nekoliko bitnijih pravila:

- Controller klasa za pojam „Xyz" će se zvati XyzController.
- Controller klasu je obvezno staviti na istu razinu s ostalim controllerima
- View datoteke vezane uz taj controller, moraju biti u mapi Views/Xyz/...
- Pri definiranju rute, ne koristi se naziv XyzController, nego samo Xyz

Nekoliko primjera pravila usmjeravanja uz pojašnjenje:

Opis	Kod
Naziv rute: Profile_default **Url**: `/moj-profil` aktivira `AccountController.Profile()`	`app.MapControllerRoute("Profile_default", "moj-profil", new { controller = "Account", action = "Profile" });`

Opis	Kod
akciju **Defaults**: samo jedan jedini URL moze aktivirati ovu akciju	
URL: `/icesar/uvod-u-dot-net` ili `/icesar`, obje rute aktiviraju akciju `BlogController.Details(string blog, string post)`. **Pojašnjenje**: Parametar **blog** se sastoji od barem jednog malih i velikih slova eng. abecede, znamenke i znaka '-'. Parametar **post** je opcionalan.	`app.MapControllerRoute(name: "BlogDetails2", url: "{blog}/{post}", defaults: new { controller = "Blog", action = "Details", post = UrlParameter.Optional }, constraints: new { blog = @"[a-zA-Z0-9-]+", post = @"[a-zA-Z0-9-]*"});`

Korisni linkovi:

- https://docs.microsoft.com/en-us/aspnet/core/fundamentals/routing?view=aspnetcore-6.0#route-constraint-reference

Usmjeravanje pomoću atributa

Od ASP.NET MVC verzije 5 dostupan je intuitivniji i lakši način definiranja ruta za controller-e i akcije – koristeći posebne anotacije. Točnije, koristi se atribut **[Route]** koji se dodaje na akciju controller-a ili na sami controller. U njemu se definira URL koji je potrebno unijeti za pristup određenoj akciji.

Primjer definiranja ruta za CityController koja se aktivira na URL `/gradovi/po-drzavi/CRO` ili `/gradovi/po-drzavi/SLO`:

```
namespace Vjezba4.Web.Controllers
{
    [Route("gradovi")]
    public class CityController : Controller
    {
        [Route("po-drzavi/{country:length(3)}")]
        public ActionResult List(string country)
        {
            //Obrada
            //...

            return View();
        }
```

```
    }
}
```

Slijedi nekoliko primjera korištenja [Route] atributa:

URL: `/Home/Index`

```
[Route("[controller]/[action]")]
public class HomeController : Controller
{
    public IActionResult Index(string lang = null)
```

URL: `/Home/Index` ili `/Home/Index/e` ili `/Home/Index/en` ili `/`

```
[Route("[controller]/[action]")]
public class HomeController : Controller
{
    [Route("/")]
    [Route("{lang:minlength(1):maxlength(2)?}")]
    public IActionResult Index(string lang = null)
```

URL: `/Home/Index` ili `/Home/Index?lang=en`

```
[Route("[controller]")]
public class HomeController : Controller
{
    [Route("[action]")]
    public IActionResult Index(string lang = null)
```

URL: `/dom/indeks`

```
[Route("dom")]
public class HomeController : Controller
{
    [Route("indeks")]
    public IActionResult Index()
```

URL: `/dom`

```
[Route("dom")]
public class HomeController : Controller
{
    [Route("")]
    public IActionResult Index()
```

URL: `/dom` ili `/dom/indeks`

```
[Route("dom")]
public class HomeController : Controller
{
    [Route("")]
    [Route("indeks")]
    public IActionResult Index()
```

Korisni linkovi:

- https://docs.microsoft.com/en-us/aspnet/core/mvc/controllers/routing?view=aspnetcore-6.0#attribute-routing

Djelomični (partial) view

Djelomični view možemo na neki način poistovjetiti s korisničkim kontrolama u ASP.NET WebForms tehnologiji – služi za iscrtavanje jednog specifičnog dijela stranice, kojeg eventualno koristimo na više mjesta u aplikaciji. Djelomični view se može koristiti na dva načina:

- **Pozivom tijekom iscrtavanja „običnog" view-a, uz proslijeđivanje odgovarajućeg modela, u samoj razor sintaksi**
- Pozivom akcije controller-a koja može vratiti PartialViewResult

Za razliku od običnog view-a, djelomični view se **ne** iscrtava pomoću _Layout stranice – kad se iscrtava „obični" view, sadržaj se ubacuje na odgovarajuće mjesto unutar _Layout stranice. Djelomični view ne uzima u obzir nikakav kontekst, već samo iscrtava sadržaj koji se tamo nalazi. Djelomični view se koristi u slučajevima kad postoji HTML/js/C# kod koji se može nalaziti na više mjesta, kako se kod ne bi kopirao i ponavljao; i samim time povećavala mogućnost pogreške.

Primjer za iscrtavanje jednog smislenog djelomičnog view-a se može naći u _Layout datoteci, kod iscrtavanja kontrole za prijavu:

_Layout.cshtml

```
<div class="float-right">
    <section id="login">
        @await Html.PartialAsync("_LoginPartial")
    </section>
    <nav>
```

U gornjem slučaju, *LoginPartial ne prima nikakav objekt kao model, i nalazi se u Shared folderu, kako bi bio dostupan u view-u. Generalno, ne postoji obvezna nomenklatura za djelomične poglede, no preporuča se korištenje gornje nomenklature – sa znakom* " (underscore) prije imena.

Od .NET Core verzije postoji i posebni asp tag kojim se iscrtava partial view, primjer kojega se može naći u _Layout stranici:

_Layout.cshtml

```
<partial name="_CookieConsentPartial" />
```

Česti primjer gdje se koristi djelomično iscrtavanje je prilikom kreirana standardnog mehanizma za kreiranje i osvježavanje podataka. Konkretno, kada dodajemo novi podatak ili kada uređujemo taj podatak, možemo primjetiti da je forma za prikaz gotovo identična – postoji eventualno razlika u tome koja akcija controller-a će obraditi zahtjev kad se zatraži prazna forma, ili kad se forma s podacima pošalje na server.

Proslijeđivanje modela u partial view

Prilikom poziva partial metode, kao i kod svakog drugog view-a, ASP.NET MVC očekuje da se proslijedi model. Primjerice, pretpostavimo da je definiran partial view:

_ClientFilter.cshtml

```
@model Vjezba.Web.Models.ClientFilterModel

<div>
```

```
    Ime: @Html.EditorFor(p => p.yName)
</div>
```

Također, pretpostavimo da imamo pregled klijenata definiran ovako:

Index.cshtml

```
@model List<Vjezba.Web.Models.Mock.Client>

@{
    ViewBag.Title = "Index";
}

<h2>Pregled klijenata</h2>
```

Postoji nekoliko načina kako bi mogli pozvati iscrtavanje _ClientFilter djelomičnog view-a:

- Pozivom `@await Html.PartialAsync("_ClientFilter")` – prouzročit će **pogrešku**, jer ukoliko ne navedemo model koji se proslijeđuje u Partial metodu, tada se automatski prosljeđuje model iz view-a koji poziva Partial metodu – u ovom slučaju proslijedio bi se kao model List<Client>, a nas view _ClientFilter očekuje kao model ClientFilterModel.
- Pozivom `@await Html.PartialAsync ("_ClientFilter", null)` – prouzročit će **pogrešku**, model koji šaljemo neće biti ispravnog tipa ali imati vrijednost „null"
- Pozivom `@await Html.PartialAsync ("_ClientFilter", new ClientFilterModel())` – šaljemo novi objekt, ovaj puta ispravnog tipa
- Pozivom `<partial name="_ClientFilter" model="new ClientFilterModel()" />`

Također, postoji opcija koristeći **RenderPartial** metodu, ali to se ostavlja čitatelju na vlastito istraživanje.

Generiranje HTML input elemenata

Od .NET Core verzije MVC-a, postoji nekoliko načina na koje možemo generirati HTML input elemente.

Naredbe EditorFor i TextBoxFor

Naredba **EditorFor** prima kao parametar **Func<>** objekt kojim se definira na koje polje se odnosi te prema tome generira HTML „name" atribut. Slično funkcionira i naredba **TextBoxFor**, ali ona uvijek generira **input type=text**, dok EditorFor generira HTML input element u ovisnosti o tipu podatka. Varijabla **p** je istog tipa kojeg je i model koji se koristi unutar view-a – u gornjem primjeru to je **ContactModel** klasa. Uz EditorFor, dostupna je i funkcija NameFor koja na sličan način generira odgovarajući „name" parametar:

Contact.cshtml

```
@model QuizManager.Web.Models.ContactModel
...
<section class="contact-form">
    <header>
        <h3>Pošaljite nam upit!</h3>
        <form action="/Home/Contact" method="post">
            <div>
                Ime: @Html.TextBoxFor(p => p.Ime)
                Prezime: <input type="text" name="@Html.Nam
eFor(p => p.Prezime)" />
            </div>
            <div>
```

Tag helper <input>

Sličnog koncepta kao EditorFor, međutim prilagođenije HTML sintaksi je tzv. Tag helper za generiranje input elemenata.

Contact.cshtml

```
@model QuizManager.Web.Models.ContactModel
...
<section class="contact-form">
    <header>
        <h3>Pošaljite nam upit!</h3>
        <form action="/Home/Contact" method="post">
            <div>
                Ime: <input asp-for="Ime" class="form-contr
ol" />
                Prezime: <input asp-for="Prezime" class="fo
```

```
rm-control" />
            </div>
            <div>
```

Može se uočiti kako asp-for atribut zapravo zamjenjuje lambda izraz (p ⇒ p.Ime), tj., možemo zamisliti kako interno asp-for izvodi identičan lambda izraz kao gore navedeno u EditorFor/TextBoxFor.

Form helper

Zadnji problem koji je još potrebno riješiti je problem ručno unešene adrese (form action atribut) na koju se šalju podaci u formi. Za to također postoji riješenje:

Contact.cshtml

```
<section class="contact-form">
    <header>
        <h3>Pošaljite nam upit!</h3>
        @using(Html.BeginForm())
        {
            ...
        }
    </header>
</section>
```

Funkcija **BeginForm** također prima niz parametara koji definiraju:

- Akciju i controller na koju se treba poslati podaci
- Method – GET ili POST
- Dodatne route parametre ili html atribute

Druga opcija koju možemo koristiti je form tag helper:

Contact.cshtml

```
<section class="contact-form">
    <header>
        <h3>Pošaljite nam upit!</h3>
        <form asp-action="SubmitQuery" method="post">
            ...
```

```
        </form>
    </header>
</section>
```

Entity framework (EF)

Pri radu s bazom podataka iz C# postoji nekoliko raznih načina kako možemo iz baze dohvatiti ili spremiti podatke:

- Korištenjem ADO.NET adaptera – set komponenti kojima se može iz C# dohvaćati podatke iz baze podataka i spremati ih u odgovarajuće prilagođene objekte (DataSet, DataTable, ...)
- Korištenjem ORM alata kao što je NHibernate, Dapper ili **EF**
    - ORM = Object Relational Mapping
    - Prikazivanje podataka iz baze korištenjem klasa u C#
    - Praćenje promjena
    - ...

Većina modernih aplikacija pisanih u .NET tehnologiji koristi Entity Framework kao alat za komunikaciju s bazom podataka.

Biblioteke i zavisne komponente

Da bi koristili entity framework u aplikaciji, potrebno je upoznati se s načinom kako se referenciraju biblioteke i zavisni projekti u .NET Core MVC Web aplikaciji. Potrabn je alat za jednostavnu manipulaciju pomoćnim bibliotekama (pluginovima), te izvođenje raznih aktivnosti vezanih uz pomoćne biblioteke, kao što je generiranje koda, izvršavanje skripti nad bazom podataka i sl. Alat koji tako nešto omogućava je Nuget package manager, i može se koristiti kroz konzolu (*package manager console*) ili UI sučelje (*Manage nuget packages for solution*).

Popis trenutno instaliranih biblioteka može se vidjeti u *solution explorer* prozoru, pod *Dependencies*.

Također, moguće je pogledati instalirane biblioteke i pakete kroz Nuget UI sučelje, desnim klikom na solution i odabirom *Manage NuGet Packages*.

Također će nam biti potrebna konzola za izvršavanje skripti vezanih uz Entity Framework (EF). Svaka naredba koja se izvodi se izvodi nad nekim projektom, što se može vidjeti u konzoli. Za rad s migracijama, koristit će se Developer PowerShell konzola.

Prije nego se krene raditi sa Entity Frameworkom, dobro je resturkturirati projekt na smislene slojeve:

- **Model** sloj – sadržavat će klase koje predstavljaju tablice u bazi podataka – primjerice Client, Meeting i sl.
- **DAL** sloj – sadržavat će migracijske skripte i klase vezane za EF
- **Web** sloj – controlleri, view-ovi, statičke HTML stranice i ostale komponente potrebne za prikaz korisničkog sučelja

Konfiguracija EF

Kroz naredno poglavlje bit će navedeni koraci koje je potrebno napraviti kako bi se EF mogao koristiti u projektu.

Prije početka osigurati da je moguće spojiti se na lokalni MSSQL server (najčešće `.\sqlexpress`). Provjeriti u SQL Server Management Studio alatu je li moguće, te dodati bazu podataka naziva ClientManager. Preporuča se koristiti docker za elegantno postavljanje baze podataka.

DbContext klasa

Prvi korak je u DAL projekt dodati klasu koja predstavlja kontekst – bazu podataka. Klasa mora **naslijediti iz DbContext** klase. U kontekstu aplikacije za evidenciju klijenata to bi bila **ClientManagerDbContext** klasa.

DAL/ClientManagerDbContext.cs

```
using System;
using System.Collections.Generic;
using Microsoft.EntityFrameworkCore;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Vjezba.DAL
{
```

```
    public class ClientManagerDbContext : DbContext
    {
        protected ClientManagerDbContext() { }

        public ClientManagerDbContext(DbContextOptions<Clie
ntManagerDbContext> options) : base(options)
        { }
    }
}
```

U `appsettings.json` u Web projektu dodati ispravne podatke za spajanje na bazu podataka. Kod može izgledati po prilici ovako, ali moguće je i da se ponešto razlikuje. Donji primjer je za spajanje na docker s MSSQL bazom:

Web projekt/appsettings.json

```
{
  "Logging": ...,
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "ClientManagerDbContext": "Data Source=127.0.0.1;Initia
l Catalog=mvc_2023_24;User ID=sa;Password=yourStrong(!)Pass
word;MultipleActiveResultSets=True;TrustServerCertificate=T
rue;"
  }
}
```

Podesiti model na sljedeći način:

- Svaka klasa iz modela (koja predstavlja redak odgovarajuće tablice u bazi podataka) treba imati polje **Id** anotirano s **[Key]** atributom
- Svaka kolekcija u modelu koja predstavlja **1-N** vezu treba biti definirana kao virtualna, tipa **ICollection<T>**
- Svaki **strani ključ** (kod 1-N veze, reference iz kolekcije) treba imati definirano **Id** polje referenciranog objekta i anotaciju **[ForeignKey]**
- Veze **N-N** se definiraju na način da se u **obje klase** koje su povezane doda **kolekcija** kako je opisano u točki 2

Donji primjeri koda su napisani za sličnu aplikaciju za evidenciju kvizova.

Models/Quiz.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.Linq;
using System.Threading.Tasks;

namespace Vjezba.Model
{
    public class Quiz
    {
        [Key]
        public int Id { get; set; }

        ...
    }
}
```

Models/Quiz.cs

```
namespace Vjezba.Model
{
    public class Quiz
    {
        [Key]
        public int Id { get; set; }

        ...

        public virtual ICollection<Question> Questions { ge
t; set; }
    }
}
```

Models/Question.cs

```
namespace QuizManagerWeb.Models
{
    public class Question
    {
        [Key]
        public int Id { get; set; }

        ...

        [ForeignKey("Quiz")]
        public int QuizId { get; set; }

        public virtual Quiz Quiz { get; set; }
    }
}
```

Models/User.cs

```
namespace QuizManagerWeb.Models
{
    public class User
    {
        [Key]
        public int Id { get; set; }

        public string UserName { get; set; }
        public string Password { get; set; }

        public virtual ICollection<Role> Roles { get; set;
}
    }
}
```

Models/Role.cs

```
namespace QuizManagerWeb.Models
{
    public class Role
```

```
    {
        [Key]
        public int Id { get; set; }

        public string Name { get; set; }

        public virtual ICollection<User> Users { get; set;
}
    }
}
```

U klasu **QuizManagerDbContext** potrebno je dodati informaciju o tome koje tablice tamo pripadaju, na način da se definira svojstvo tipa **DbSet<T>**, gdje je T tip klase koja predstavlja redak u nekoj tablici. Dodati **DbSet<T>** za svaku klasu/entitet koji želimo da bude tablica u bazi podataka.

/QuizManagerDbContext.cs

```
using QuizManager.Model;
using System;
using System.Collections.Generic;
using Microsoft.EntityFrameworkCore;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace QuizManager.DAL
{
    public class QuizManagerDbContext : DbContext
    {
        public DbSet<Quiz> Quizzes { get; set; }
    }
}
```

Registrirati SQL Server i EF u Program.cs datoteci.

Program.cs

```
builder.Services.AddDbContext<ClientManagerDbContext>(optio
ns =>
```

```
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("ClientMa
nagerDbContext"),
        opt => opt.MigrationsAssembly("Vjezba.DAL")));
```

Migracijske skripte

Kako bi EF mogao ispravno funkcionirati, shema baze podataka mora biti u skladu s klasama definiranim u projektu. Održavanje sheme baza i modela klasa u sinkronicitetu nije uvijek jednostavan zadatak zbog kontinuiranih promjena modela, pogotovo ako se radi ručno, bez dodatne pomoći specijaliziranih alata. Srećom, EF sadrži alat za usklađivanje sheme baze i modela: *EF migrations*.

Općenito, pri svakoj promjeni modela (može biti više promjena odjednom), potrebno je provesti sljedeći proces kako bi se EF i baza uskladili:

1. Napraviti željene promjene u modelu (primjerice, dodati novo polje, novu klasu, itd.)
2. Dodati migraciju proizvoljnog opisnog imena naredbom `dotnet ef migrations add NAZIV`
3. Osvježiti bazu pozivom naredbe `dotnet ef database update NAZIV`
    a. Ukoliko se radi o bazi podataka koja je u produkcijskom okruženju, moguće je i pozvati naredbu `dotnet ef migrations script FROM TO` koja odgovarajuću skriptu generira u .sql datoteku, te se može naknadno izvesti ručno na željenoj instanci baze podataka putem MSSQL management studio alata ili ServerExplorer prozora.

Dodatno, treba paziti da se u višeslojnoj aplikaciji ispravno definiraju parametri koji određuju gdje se model nalazi, gdje se nalazi DB context klasa te gdje se nalaze podaci za spajanje na bazu (*connection string*). Za to su zaduženi sljedeći parametri:

- **--startup-project XXXX** – definira koji projekt sadrži ispravne connection string
- **--context XXXX** – koja klasa se uzima kao DB context
- Projekt u kojem želimo migracije mora biti „root" projekt naredbe u developer konzoli.

Isti parametri koiste se kod svih gore navedenih naredbi. U praksi, kreiranje inicijalne migracije može izgledati ovako:

Developer PowerShell

```
C:\git-tt\net-ok-23-24\Vjezba.DAL> dotnet ef migrations add
Initial --startup-project ../Vjezba.Web --context ClientMan
agerDbContext

Build started...
Build succeeded.
The Entity Framework tools version '6.0.12' is older than t
hat of the runtime '8.0.4'.
Update the tools for the latest features and bug fixes. See
https://aka.ms/AAc1fbw for more information.

Done. To undo this action, use 'ef migrations remove'
```

Osvježavanje baze podataka bi izgledalo ovako:

Developer PowerShell

```
C:\git-tt\net-ok-20-21\Vjezba.DAL> dotnet ef database updat
e --startup-project ../Vjezba.Web --context ClientManagerDb
Context
```

U „real-world" projektima, česta je praksa definirati migrations-readme datoteku u kojoj se nalaze gornje naredbe s već definiranim svim potrebnim parametrima, te se novi članovi tima mogu lako snaći.

Napomena: ako dobijete grešku da startup projekt ne referencira Microsoft.EntityFrameworkCore.Design, potrebno je probati rebuild projekta (build → rebuild all).

Inicijalni podaci

Pri razvoju projekata česta je situacija (pogotovo na početku) da su nam potrebni nekakvi podaci s kojima možemo testirati ostale funkcionalnosti aplikacije. Također, možemo napuniti i neke potrebne podatke kao što su razni tipovi, lookup vrijednosti i slično koji ostaju i u produkcijskom okruženju. U EF core alatu, inicijalni podaci se pune na način da se unutar modela proširi funkcionalnost metode OnModelCreating unutar koje se definiraju podaci za pojedine entitete.

ClientManagerDbContext

```
protected override void OnModelCreating(ModelBuilder modelB
uilder)
{
    base.OnModelCreating(modelBuilder);

    modelBuilder.Entity<City>().HasData(new City { ID = 1,
Name = "Zagreb" });
    ...
}
```

Nakon toga, potrebno je izgenerirati novu migraciju i pokrenuti kako bi se podaci uspješno osvježili.

Korisni linkovi:

- https://docs.microsoft.com/en-us/ef/core/modeling/data-seeding

CRUD operacije

C: Create – stvaranje podatka u bazi

R: Read – čitanje podataka iz baze

U: Update – promjena postojećih podataka u bazi

D: Delete – brisanje odgovarajućeg podatka iz baze

U ovom poglavlju ćemo pogledati kako iskoristiti EF za svaku od pojedinih operacija nad bazom podataka.

Rukovanje kontekstom

Kako bi mogli koristiti EF, potrebna nam je instanca DB context klase, i upravo taj objekt prati promjene koje radimo nad entitetima i sprema ih u bazu podataka (kad dodajemo nove, mijenjamo postojeće i sl.). Jedan od izazova je definirati trajanje i životni ciklus upravo tog objekta.

Postoje dva načina kako se može kreirati instancu DB context klase: ručno, pozivom „new" ili pustiti MVC radni okvir da ju kreira.

Kreiranje DB context klase ručno

```
var context = new QuizManagerDbContext();

//CRUD operacije nad kontekstom

context.Dispose();
```

Automatsko kreiranje instance

```
public class QuizController : Controller
{
    private QuizManagerDbContext _dbContext;

    public QuizController(QuizManagerDbContext dbContext)
    {
        this._dbContext = dbContext;
    }
```

Nakon toga koristimo `this._dbContext` kad god nam treba.

Preporučeni način korištenja je automatskim kreiranjem, i u ovom trenutku neće se dublje ulaziti u sami mehanizam po kojem to funkcionira. Navedeni mehanizam je Dependency Injection, te ga je moguće samostalno istražiti, a detaljnije će biti objašnjen u sklopu kolegija Napredni web servisi .NET na specijalističkom studiju.

Create – stvaranje podataka

Kako bi kreirali podatak u bazi, koristimo sljedeći set naredbi (pod uvjetom da postoji ranije kreirani obični objekt tipa Quiz):

```
//Dodavanje novog kviza (pretpostavimo da postoji varijabla
Quiz quiz)
this._context.Quizes.Add(quiz);

//Spremanje promjena (commit)
this._context.SaveChanges();
```

Read – dohvaćanje i manipulacija podacima

Za dohvaćanje podataka koriste se LINQ upiti koji se pri upitu prevode u SQL naredbe, izvršavaju na bazi i kao rezultat vraćaju set objekata koji želimo dohvatiti. U načelu, naredbe za čitanje dijelimo na:

- naredbe za dohvat jednog podatka (po primarnom ključu)
- naredbe za dohvat više podataka (pretraga po kriteriju)

Za dohvaćanje jednog podatka možemo koristiti jedan od dva odsječka:

```
//U ovom slučaju ostavljamo mogućonst da quiz s željenim id
-jem ne postoji
//Varijabla id sadrži vrijednost prema kojoj želimo dohvati
ti kviz
Quiz result = this._context.Quizes
    .Where(p => p.Id == id)
    .FirstOrDefault();
```

```
//Dohvaćanje putem poziva funkcije 'find'
Quiz result = this._context.Quizes
    .Find(id);
```

Za dohvaćanje više podataka koristimo željene LINQ naredbe koje su obrađene u vježbi 3. Primjer dohvaćanja svih kvizova koji su nastali u 2013. godini:

```
//Dohvaćanje svih u 2013.
List<Quiz> result = this._context.Quizes
    .Where(p => p.DateCreated.Year == 2013)
    .ToList();
```

Tek pozivom **ToList()** naredbe se upit zaista prevodi u SQL i izvršava na bazi podataka.

Često postoji potreba da se iz baze automatski dohvate i relacije pojedine tablice. Primjerice, ukoliko kviz ima kategoriju (1-N veza), ovakav kod bi bacio iznimku, jer relacijsko svojstvo Category zahtjeva ili da se naknadno dohvati iz baze, ili da se učita automatski u samom upitu:

```
public IActionResult Details(int? id = null)
{
```

```
    var quiz = this._dbContext.Quizes
        .Where(p => p.ID == id)
        .FirstOrDefault();

    var categoryName = quiz.Category.Name;
    return View(null);
}
```

Kako bi to popravili, potrebno je već pri učitavanju kvizova dohvatiti i kategoriju (napuniti relacijsko svojstvo):

```
//Dohvaćanje svih u 2013.
List<Quiz> result = this._context.Quizes
    .Include(p => p.Category)
    .Where(p => p.DateCreated.Year == 2013)
    .ToList();

var categoryName = result.First().Category.Name; // OK
```

Napomena: funkcija Include se nalazi u imenskom prostoru **Microsoft.EntityFremeworkCore**. Također, ne treba pretjerati sa uključivanjem puno relacija jer time upit postaje kompleksniji i performanse se mogu drastično smanjiti.

Update i delete – izmjena i brisanje postojećeg podatka

U ovoj fazi neće se raditi update/delete primjeri, ali slijede odsječci koda pomoću kojih se navedene operacije mogu napraviti:

```
//Dohvat željenog podatka
Quiz result = this._context.Quizes
    .Find(id);

//Primjer izmjene
result.Title = "Izmjenjeni naslov";

//Spremanje promjena (commit)
this._context.SaveChanges();
```

```
//Dohvat željenog podatka
Quiz result = this._context.Quizes
    .Find(id);

//Postavljanje stanja na deleted - označavanje da je kviz o
brisan
this._context.Entry(result).State = System.Data.Entity.Enti
tyState.Deleted;

//Spremanje promjena (commit)
this._context.SaveChanges();
```